Wins and Accomplishments since last quarter

@John Thomalla **– Interop Team**

Win - What did we achieve?

Self Service Application - An application designed to enable on-demand configuration changes, updates, and execution of implementation and setup tasks that traditionally require developer involvement.

This solution minimizes development overhead and associated costs by reducing reliance on engineering resources for routine operations. It achieves this through the automation of repetitive processes and the standardization of operational workflows. Additionally, it empowers internal stakeholders, including Support, Implementation, and Customer Success teams to independently and efficiently perform tasks, thereby improving scalability, accelerating turnaround times, and optimizing resource allocation across the organization.

The Self Service Application is built on a modern, full-stack JavaScript/TypeScript architecture designed to support scalable, maintainable, and efficient delivery of on-demand configuration and operational workflows.

The front-end layer leverages **Next.js**, **React**, and **TypeScript** to deliver a performant, responsive, and type-safe user interface. This combination enables server-side rendering and optimized client-side interactions, resulting in improved load times, enhanced user experience, and increased developer productivity through strong typing and component reusability.

The back-end is implemented using **Node.js** with the **Express** framework, also written in **TypeScript**, providing a lightweight and extensible API layer. This architecture supports the orchestration of business logic, execution of automated tasks, and management of

configuration changes, while ensuring consistency and reliability through type safety across the full stack.

By standardizing on TypeScript across both client and server, the platform benefits from improved code quality, easier maintainability, and reduced runtime errors. Overall, this technology stack enables rapid development, efficient scaling, and reduced development overhead by streamlining the implementation of automated workflows and minimizing the need for direct engineering involvement in routine operations.

What Improved - What's the Proof?
Still measuring and collecting data.

What do we know now that we didn't then?

@Damien Evans **– OnePacs**

FDA Audit Remediation

@Rob Giordano **– EMR**

Performance and Stability - HCA explicitly stated that EMR stability and performance in Q1 were *"the best they've seen in the past two years."* This feedback was shared after a meeting and relayed internally as direct praise for the improvement work. Also, a HCA provider posted in a HCA Webex chat indicating they are seeing "*significantly fewer error messages in day-to-day use" after the release of 26.1.*

Tyler and Zach moving to Team Montana "full time".

Zach Pollard stepping up to help fill the gap with recent departures, also taking ownership of prod incidents.

The completion of the **Decommissioning Tool,** now DEV is not needed to babsit that process.

@Henry Happ **– DevOps**

EMR PR Gates: Fulfilled a request from Scott Seely to update implement PR checks that require all unit tests to run successfully prior to approving a PR for merging

EMR Remoting HealthCheck Service: Implemented the CI/CD tooling to get the service deployed and operational on the remoting endpoint servers.

Tempus Analyzer: Created build/deploy process using update code signing certificate process including an updated certificate installation process provided by Tempus. Follow up included working with Support and ?? for implementation process and customized installation for HCA's requirements.

@Isaac Blatter **– Team Montana/AI Scribe**

What value have I contributed?

System stabilization

- fixed dozens of recording/transcription issues, all due to Apple bugs, (we have almost no QA for any apple products)
- whisperx to dramatically reduce load and costs
- dozens of stability enhancements (allowed us to reduce our app instance count)

Infrastructure refactoring

- improving/standardizing AWS deployment pipeline

Agentic re-architecting

- shift to agentic architecture
- constant iteration as we learn more (e.g. controlling tool/mcp use)
- allowing more features: user interaction, shift from batch processing

Product interaction re-design

- creating a full end-to-end feature tracking from milestone down to individual stories
- dev ownership of Jira backlog, product ownership of PRD's and roadmap

What am I going to contribute? (working on now)

Working with the team

- continuously re-evaluate how we work with AI (agentic configuration, etc.)

Understand how our systems behave in the wild

- clinic visits to understand how our users interact with our system
- how can we have one place to constantly monitor everything
- alarms for key pieces

How can I drive adoption of scribe

- define metrics on how we've improved workflows

- how to collect better feedback

Automating quality (shift quality left)

- automated testing
- AI tools for QA
- improving unit test coverage
- how to test end to end before dev

Development metrics discovery

- metrics for quality, not just story points
- how to improve sprints (no end to end

@Rafael Van Dyke **– PM/PE/RCM**

What did we achieve?

- Backfill for Vini, replaced Loka & GenUI with Mistura
- PEgasus & OptiPMizers sustained completion rates after split from Agile Avengers
  - Sprint Predictability & Sprint Completion Rates > 90%
- IText: Cost restructure; replacement underway
  - Thank you, Henry & Isaac!!
- Promotion for Ashwin to Senior Engineer
  - The cross-functional support, unblocking teammates, working with product and DevOps, work on Payments
- We reinforced our expectations around AI adoption and core values, ensuring full alignment across the team as we scale our AI-driven strategy.
  - Velocity increasing on Bill-iant Minds with AI tool, which Dylan will demo for us in an upcoming SEL Weekly Hour.

What improved, and what's the proof?

- Too early for me to determine, but I have ideas for next time!

What do we know now that we didn't then?

- CA Core & Alpha were not agile and not delivering
  - Replacements for Loka & GenUI underway
- New PM, Nick, already establishing foundation for productivity and focus on MVP.
- Core Stability point of emphasis on root cause fix over data fixes
  - Adjustment on Green Initiative volume for Rocinante

- Sprint Addition Rate metrics not up to industry standard of < 30%
  - Previous goal for Pegasus: <60%, currently sitting at 40%
- Need more clarity on what Dylan and Heath are supposed to do, and how their performance should be measured
  - Are they architects, lead engineers, or something else?
  - Assuming they're not architects, how does their impact look differently than that? What specific value do they bring other than substituting for lack of resources on specific projects that are urgent?

@Scott Seely **– Platform Engineering/Core Stability**

Wins:

- Importance of onboarding new contractors and getting them productive ASAP has reached org level.
  - Improved:
    - Lists of apps are known. Acknowledged that we need to go further.
    - Single "common" view for a dev is known across products.
    - Cross project dependencies have early documentation as generated by some more complex AI interactions.
  - Know now that we didn't before:
    - Containerization of "all the things" is difficult on a Windows VM under current restrictions.
    - Proper path to containerization may be simpler than first thought. No need for dozens of containers to handle the APIs and such; more likely that we have a few "conglomeration containers"
    - Need for "clean" database set is crystal clear to all. Need to generate this set soon. Make it easy to backup, restore, etc.
- EMR and PM are taking good changes, blocking things because stability isn't there.
  - Improved:
    - Very rigorous testing of the WebSocket functionality. Great to see the push to get it stabilized and not accept "small issues".
    - Prioritization of making changes to improve serviceability has happened.
    - VOC is more visible and acted upon.
  - Know now that we didn't before:

- Dependencies that have had bug fixes need coordinated updates. We know the scope of things.
- The “just switch to WCF” isn’t so clear cut. We’ve uncovered a number of unknowns that are now known and being figured out.